CS482/495/496 Software Project Proposal: add your tentative project title here

Fabrizio Guzzo, Dylan Morales, Will Troisi, Zoe Willis

2025-11-24

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Prof. Ebert
- Client title: Professor
- Client email address: eebert@loyola.edu
- Client employer: University Of Loyola Maryland
- How you know the client: Professor at Loyola

2 Project Description

2.1 Overview

In this project, we are making an application for a baseball league. This application is for both managing and viewing; including various roles such as player and admin. The problem we are trying to solve is that the league does not have any web-based application that stores data and is easy to manage. We want the baseball league to have an application that allows coaches/managers to control their team as well as an website that allow players to view upcoming games and other important information.

2.2 Key Features

While our client gave us a lot of features to add, we will only list out the key features. The basics overview of the project is creating a piece of software that will run on the web that will manage the teams, coaches, players, etc. There is a login/signup for users. There should be a rating system for coaches and managers, on a scale of 1-5, which any user can use to rate. There should be a live scoreboard for the game somewhere on the app as well as a live chat room. The landing page should have a feed for comments, pictures, and videos posted by registered users. There should be a calendar with tagged matches (normal, playoff, etc.). Admins can promote users to managers and there must always be a manager for a team. Although there is a login system, there should also be a way to be logged in as a guest. Guests have restricted privileges; they can view live chat rooms but not type.

2.3 Why this Project is Interesting

While we did not choose this project for CS482, it was still very important. I believe a baseball management project that uses a database and a web application is a very important project for developers to do. It gives a lot of insight into real-world application development which is important for students to experience as it much closer to what their job could be like compared to previous coding assignments.

2.4 Areas of CS required

The areas/subfields of computer science that seem most likely to be relevant to this project are software engineering, web development, database management, human-computer interaction

2.5 Potential Concerns and Questions

My and my teammates are very inexperienced in javascript, so we are worried about starting off the project. Once we setup the environment and get through the first iteration, we believe it will be a lot more manageable. We have also never worked with mongodb or any database application, so this could be a problem if we don't figure it out early on. Personally (Will) I am worried that we will not have the project done in time. While completing each iteration by user story points seems good, I am worried that the NFR's and other basic functionality will be neglected, leaving a lot of issues and features left out due to not focusing on them.

3 Requirements

3.1 Non-Functional Requirements

ID	NFR Title	Category	Description
U0	Color Scheme	Usability (Visual Design)	Our color scheme for the application will be orange and black
S0	User Privileges	Security	Each user should have their own privileges and not be able access privileges than they deserve
U1	Navigation	Usability	Every file should be straight forward and easy to navigate with a home button and a navbar
M0	Database Storage	Maintainability	The database should maintain data even when server is shut down
S1	Sessions	Security	Application should have "sessions" where users stay logged in even when visiting different pages
M1	Testing	Maintainability	The code should be tested to ensure that the website doesn't run into errors and crash
U2	Style Conformity	Usability (Visual Design)	Our application should follow the same styling including button placement, color scheme, etc.
M2	Correct logging	Maintainability	Error messages and other console output should be outputted correctly and easily understood
U3	Focuses on essentials	Usability	Design should not be cluttered/hard to read; each file should be clear in what its purpose is.
S2	Data Privacy	Security	Only display necessary information; don't display private information if not needed

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

Table 2: Functional requirements as User Stories.

ID	Story Title	Points	Description
U0	Create User	2	As a User, I want to be able to create an account, so that I can store my information.
U1	Viewing Scores	2	As a User, to be able to view current and previous games, so that I can see all games.

ID	Story Title	Points	Description
U2	Add a child	2	As a User, I want to be able to add a child under my name, so that I can register my child as a player.
U3	Landing Page	2	As a User, I want to be able to view a landing page, so that I can navigate throughout the application.
U4	User can see/accept/decline invites	2	As a User, I want to be able to view the live chat room, so that I can post/comment in it.
U5	Rating System	2	As a User, I want to be able to rate a Manager/Coach, so that I can share my opinion.
U6	View Ratings	2	As a User, I want to be able to view ratings of Managers/Coaches, so that I can view how good they are.
U7	Change Ratings	2	As a User, I want to be able to change my rating, so that I can update my rating.
U8	Delete Ratings	2	As a User, I want to be able to delete my rating, so that I am not forced to have a rating in case I give the wrong person a rating.
U9	Chat Box	8	As a User, I want to be able to use a chat box, so that I can share my opinion about the game as well as see others.
C0	Update Score	2	As a Coach, I want to be able to update the score, so that users are able to view the current scores of games.
C1	Accept Team Invite	2	As a Coach, I want to be able to see the invite, so that I can accept it and join the team.
ADM1	Create Game	2	As an Admin, I want to be able to create a match between two teams so that they are displayed within the webpage.
ADM2	Delete Game	2	As an Admin, I want to delete individual games or the entire game history so that I can clean the database.
ADM3	Read User	2	As an Admin, I want to view a user so that I can see who is in my database and verify or make changes.
ADM4	Delete User	2	As an Admin, I want to delete a user so that I can remove them from the database.
ADM5	Update User	2	As an Admin, I want to update a user so that I can make changes when necessary.
ADM6	Add Game to Calendar	2	As an Admin, I want to add games to the calendar so that users can view the scheduled games.
ADM7	View All Teams	2	As an Admin, I want to see all the teams that are created so that I can view what teams exist in the database.
ADM8	Remove Games From Schedule	2	As an Admin, I want to remove games from the schedule so that postponed or canceled games are reflected properly.
ADM9	Delete Team	2	As an Admin, I want to delete a team so that unwanted teams can be removed from the database.

ID	Story Title	Points	Description
ADM10	Create User	2	As an Admin, I want to create a user so that new users can be added to the database.
ADM11	Invite to Manager	1	As an Admin, I want to invite an adult to be a manager so that each team has an assigned manager.
ADM12	Admin Page	2	As an Admin, I want an admin page so that I can easily manage teams, users, and other admin-only tasks in a single location.
ADM13	Game Type	2	As an Admin, I want to tag each match as normal, playoff, or final so that users can easily understand the importance of each game.
ADM14	Update Game	2	As an Admin, I want to be able to update each game, so that I can change the score
ADM15	View Game	2	As an Admin, I want to be able to view games, so that I can see all games
M0	Create Team	2	As a Manager, I want to create a team so that I can have a team with a name and logo.
M1	View My Team	2	As a Manager, I want to view my team so that I can see my players, coach, record, and schedule.
M2	Add Player to Team	2	As a Manager, I want to add and remove players from my team so that I can make changes to my roster.
M3	Invite a Coach to Team	2	As a Manager, I want to invite a coach to my team so that the coach can receive an invitation.
M4	View Players	2	As a Manager, I want to view a list of players so that I can choose who to invite to my team.
T1	Polishing of Landing Page	2	As a Technical User, I want the landing page to have improved UI so that I can navigate the site more easily.

4 System Design

4.1 Architecture

Our team has chosen to utilize the MVC structured architecture to allow for obvious focus point on our development. Using MVC we can split up our work into 3 main categories: Model, View, and Controller. Using this style of structure will promote scalability, mendability, and maintenance as we move further down the road. It will allow for developers to handle different parts of the project, avoiding the conflict of waiting on another group member to finish before starting your work. When it comes to tools we will use for each part of MVC that will be explained in our Technology section. The module will be the main branch that handles our data that will contain important information for the game consistently and securely. The controller is the link between the model and the view. It will take in requests data from the user and retrieve data from the model to display on the view. The view is the display of the structure and it allows for the user to see the information our software is displaying. Many benefits will be created from using this structure that will allow for efficiency, debugging, organization, modular development, team collaboration, and maintainable software.

4.2 Diagrams

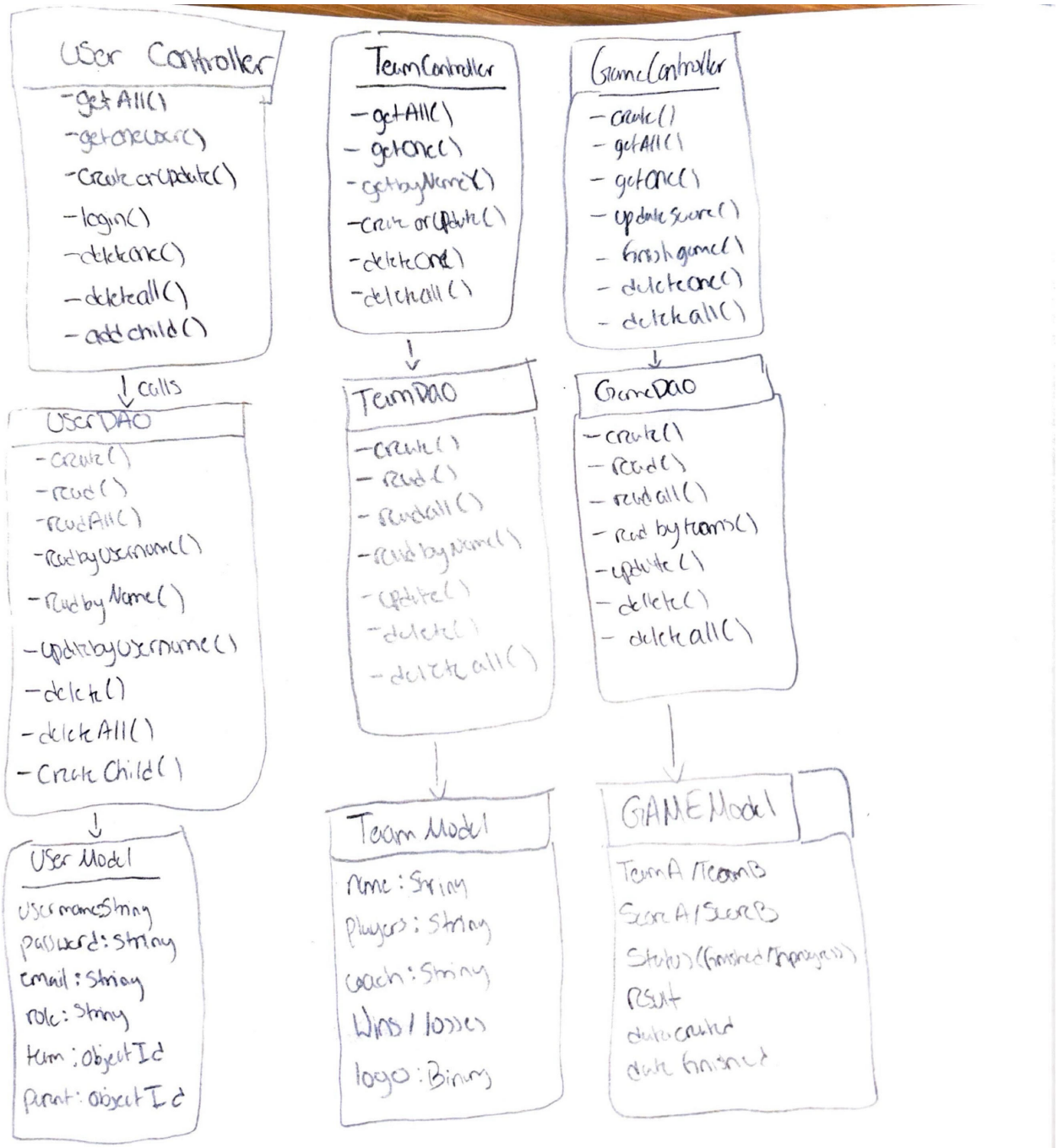


Figure 1: Coverage report

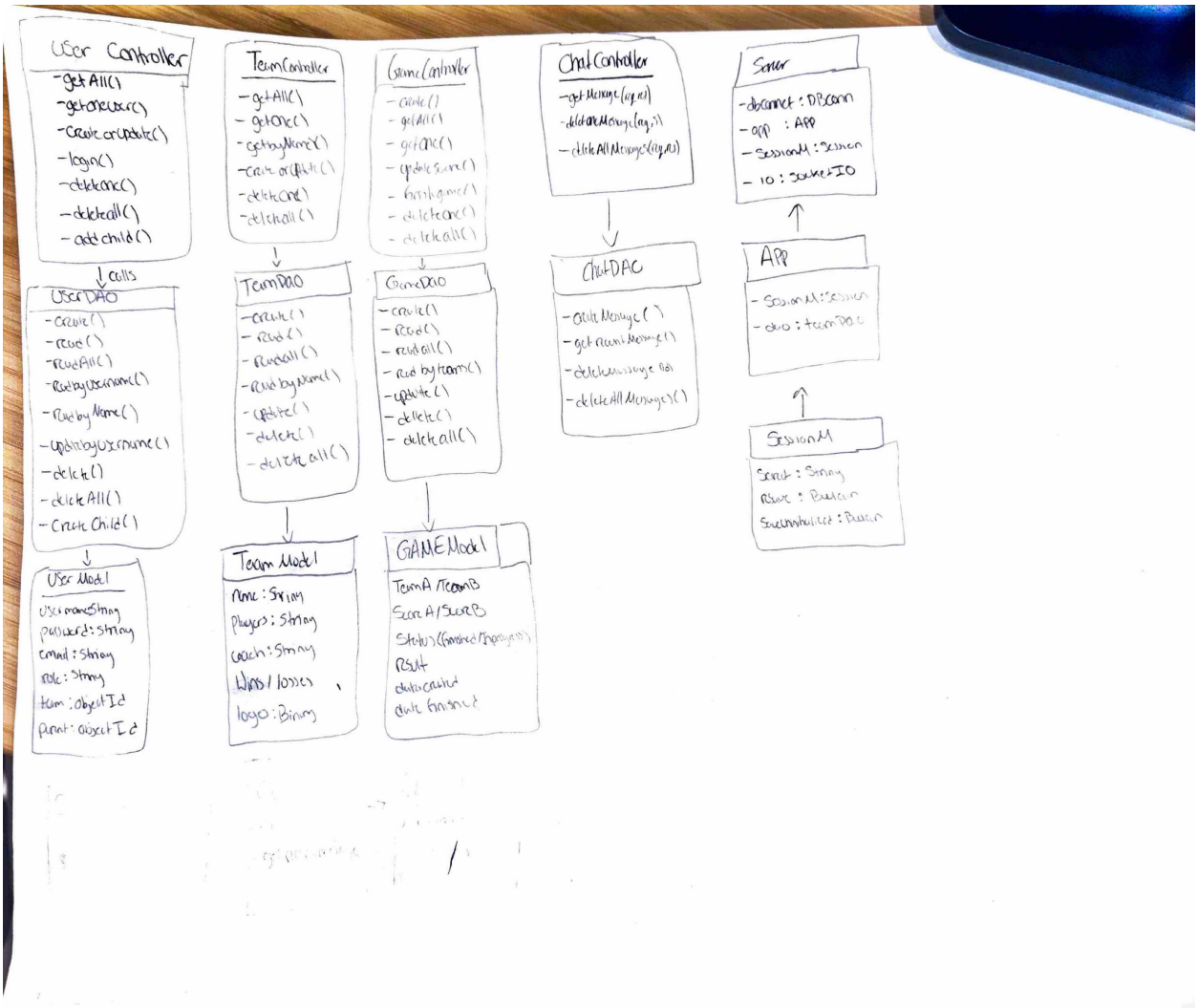


Figure 2: Coverage report

4.3 Technology

For the front-end, we will use React to handle with the user interface. In terms of our back-end we will use JavaScript to handle the interaction of the front-end with data. For API usage, we are going to utilize NodeJS to gather the necessary information to display on our website.

4.4 Coding Standards

-We are going to limit the use of global variables. -Header Format of each different Moduel: Name Date Author Synopse Variables -We are going to utilize CammelCase for Variables and Functions -Final Code Shouldn't be pushed unless tested beforehand or explicitly given noticed of an issue -Comments must be made in order to help entire team understand code

4.5 Data

Logical Model of the database entities and their relationships:

Adult (id_Adult; Name; Dob; Email; Phone; Permissions;);
Player/Child (@id_player; #id_Adult; #id_Team; Dob;);
Coach (@id_Coach, #id_Adult; #id_Team;);

Team (@id_Team; #id_Coach; #id_Manager; Playerlist;
);
Guest (@id_Guest; Permissions;);
Admin (@id_Admin; #id_Adult;);
Manager (@id_Manager; #id_Team; #id_Adult;);
Parent (@id_Parent; #id_Adult;);

4.6 UI Mocks

UI Mocks.pdf

UI Mocks.pdf

5 Iterations

5.1 Iteration Planning

5.2 Iteration/Sprint 1

5.2.1 Planning

Will: P0 Establishing Database 1pt. U1 Rate my manager 1pt. Manager invites coach 1pt. Manager Invites player 1pt.
Zoe: U11 Landing page 2pts. G0 Guest Page2pts
Fabri: U3 Login/Guest 1pt, U2 Create Scoreboard 1pt, U3 standings 1 pt, u6Color scheme 1pt
Dylan: u10 Create team 1pt, ADM5 Admin Page 2 pts, ADM1 Season Lockings 1 pt
Altered Stories

Iteration	Dates	Stories	Points
1	10/09 - 10/23	ADM10(Create User), ADM4(Delete User), U3 (Create Landing Page) G0(Login as Guest), M0(Create Team), M1(View Team), ADM14 (Update Game), ADM15(View Game),	16
2	10/23 - 11/06	ADM5(Update User), ADM3(Read User), C0 (Update Score), ADM1(Create Game), ADM2 (Delete Game), M3 (Invite a coach to a team), ADM9(Delete Team), M4(View Player), U3 (Landing Page), U2(Add a child), M2(Add player to team)	22
3	11/06 - 11/20	U5(Create Rating System), U6(View Rating) U7 (Update Rating), U8 (Delete Rating), ADM11(Invite a Manager), M3(Invite a coach to team), C1(Accept Team Invite), U4(See Invites), U9(Chat box), G0(Login as Guest), T1(Beautification/Polishing of Web Design GUI), ADM13(Game Type)	27
		Total:	65

Table 3: Iteration Planning for Incremental Deliveries

Will: ADM10(Create User CRUD) [2], ADM4(Delete User CRUD) [2] Total: [4]
Dylan: Create a Team, View Team (4 pts)
Fabri: Created a game, Update Game(Game Status and Score), Save Game(CRUD =4pts).
Explored the use of react, while not implemented in terms of the Game method, We still have a basis template to then implement with our html pages.

5.2.2 Work Done

The stories we completed on this iteration were
Dylan - create a Team and view Teams
Will - Create a user
Zoe -
Fabri - create Game Functionality
Partially completed stories
Dylan - Using React
Will - Delete a user
Zoe -
Fabri - React front end Implementation

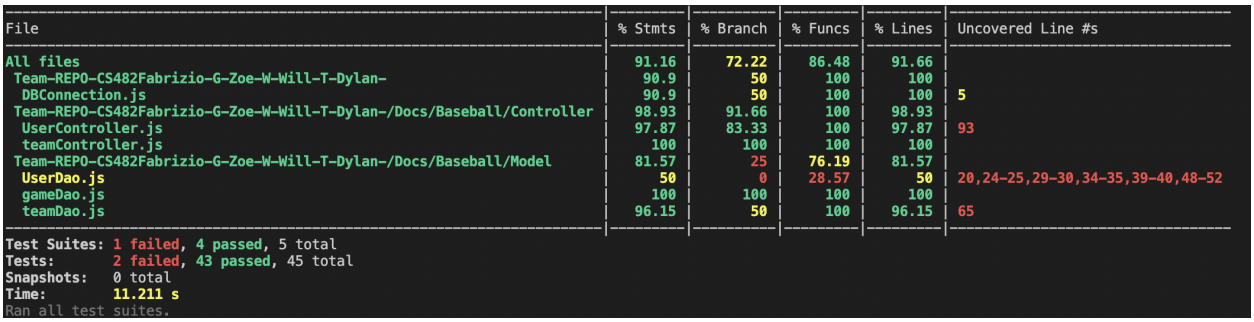
In detail:

Dylan- Created a website that creates a team using the field's name, manager, and logo. The website then uses the controller and DAO for team to store the information in the database. The website can also display all the teams that are in the database using the controller and dao.

Will - I implemented create user functionality completely. You can create a user and checks for users with the same name (the same username is not allowed twice. I did not complete delete user completely as there was no "read users" menu to select which user to delete, the "admin" would have to manually type in the username to delete it. Will fix in next iteration

Zoe -

Fabri - Implemented full game functionality including creating new games, updating existing game information(status and score), and saving completed games covering all core CRUD operations(4 pts). Additionally, we explored integrating React for potential frontend expansion. While React was not directly



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	91.16	72.22	86.48	91.66	
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-	90.9	50	100	100	
DBConnection.js	90.9	50	100	100	5
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-/Docs/Baseball/Controller	98.93	91.66	100	98.93	
UserController.js	97.87	83.33	100	97.87	93
teamController.js	100	100	100	100	
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-/Docs/Baseball/Model	81.57	25	76.19	81.57	
UserDao.js	50	0	28.57	50	20,24-25,29-30,34-35,39-40,48-52
gameDao.js	100	100	100	100	
teamDao.js	96.15	50	100	96.15	65

Test Suites: 1 failed, 4 passed, 5 total
Tests: 2 failed, 43 passed, 45 total
Snapshots: 0 total
Time: 11.211 s
Ran all test suites.

Figure 3: Coverage report

implemented for the Game module at this stage, we successfully established a foundational template that can be connected to our existing HTML pages in future iterations. This groundwork ensures an easier transition to a more dynamic, component-based user interface later on.

5.2.3 Testing Coverage

The overall coverage is pretty good at around 91, which means most of the code is being tested. The controllers and DAOs have high coverage, but UserDao.js is still low since some of its database and error-handling parts aren't tested yet. The branch coverage is also lower because we didn't include admin or error checks in this version. To improve, we'd add more tests for different CRUD cases and edge scenarios. For now, this level of coverage is solid and shows that most of the main functionality works as expected.

5.2.4 Retroespective & Reflection

The challenges we faced initially began with our user stories. We found them to be to vague/general so when it came time to our first sprint, each story we had thought to be rather simple, turned out to be worth more crud pts. In order to avoid the same mistake we should make sure that the user stories we take on have a clear narrative and point worth.

Dylan - Something that I learned is that it is important to have a strong and correct structure of MVC to make sure everything is easy to maintain and change. Also React is very new to me and I need to make sure I get a good understanding, so implementation will be easier.

Fabri- I'm currently in the process of learning how to effectively integrate React with backend database operations, specifically focusing on CRUD functionality. This experience has been valuable for helping me understand how data flows between the frontend and backend, along with how user interactions can trigger database updates. Working directly with database logic has also improved my confidence in handling database work. Overall, I'm gaining a stronger grasp of fullstack development and how each layer contributes to building interactive applications.

Will - I'm sitll fairly new to using javascript, so creating the server as well as completing my user stories proved to be very tedious. The actual coding of my user stories was not as difficult as I orignally thought it would be and in the past, I've always tried to avoid javascript. I am now realizing that it is not as bad as I originally thought. Overall, I do not think this project will be as hard as I originally thought. As long as we fulfill the requirements each iteration, this should be manageable as a project.

5.3 Iteration/Sprint 2

5.3.1 Planning

Fabrizio: C0 [1], ADM1 [1], ADM2 [1], U1 [1]. Each of these work on the backend part, but not the front end. I chose this for my Iteration 2 planning as it will complete the CRUD for Games. Total Story Points = 4. Following Will, once I've completed these I will work on other user stories.

Will: ADM5 (Update user CRUD) [2], ADM4 (Delete user CRUD [1], ADM3 (Update user CRUD) [2]. ADM3 was partially completed, so only 1 point left for that. I chose this for my iteration as it will complete

the CRUD for the Users. Total Story Points = 5. Depending on when I complete this, I may complete/work on more user stories.

Dylan: M3 [2], ADM9 [2], M9 [1] (not sure if counts as a point), M1 [1]. M9 will need the implementation of adding a file (photo) for the team logo when creating the team. It also needs a color scheme update. M1 will just need to display that logo when showing the teams. M3 will be inviting a coach to a team, so it needs to read off the coaches and have an invite button to send a message. ADM9 gives the admin the ability to delete a team if needed. Total points = 5. If this is completed and I have more time, I will invite a player system.

Zoe: U3 [2], U2 [2], M2 [2].

5.3.2 Work Done

Fabrizio: During this sprint I worked on manipulating the front end aspect of the web application for viewing and updating games. During the previous sprint I had fully completed the back end side of the CRUD operations for utilizing these features, but didn't fully completed the front end in order to perform all functionality. I also Worked on some minor tweaks in terms of utilization as a user such as navigation and overall visual appeal.

Dylan: For this iteration I made sure that I fixed my problem from last iteration. The problem was that when I made a team it did not insert an image for the team. Once that was fixed I finished the rest of my full crud for team by creating ways to update and delete teams from the data base. I also edited the color scheme to black and orange to match the wanted colors for this project.

Will: For this iteration, I focused on completing the CRUD for users and making viable html pages to interact with these functions. I finished up delete from the last iteration and finished the read and update portion. Last iteration, I had it so that you could only delete a user if you knew their username (primary key) and inputted it yourself, rather than having a list to select from. By completing the read portion of CRUD it made making the updated deleteUser html page very simple.

Zoe: During this iteration I focused primarily on enhancing the user interface and integrating new backend functionality that connects users to their related features. I completely redesigned the landing page by adding proper redirection and navigation for most of the buttons. I also added a dynamic calendar that displays games by day once they have been completed. The landing page is still a work in progress since more functionality will be connected here as features are implemented. On the backend, I spent considerable time developing the ability for parents to add a child. This required significant updates to the controller and DAO layers. Once a parent signs in, they are now able to register a child account under their profile. Additionally, I implemented age restrictions to ensure that only eligible users can sign in. I also completed story M2, allowing managers to add existing players to a team through the "edit team" page.

5.3.3 Testing Coverage

A lot of the tesing was completed during our last sprint, however userDao was low so we focused on getting all of it done. More testing for frontend will be needed later on but this testing is for backend.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	85.95	61.84	93.87	86.24	
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-	81.81	50	75	90	
DBConnection.js	81.81	50	75	90	9
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-/Docs/Baseball/Controller	81.77	62.5	95.23	81.77	
UserController.js	81.7	63.15	87.5	81.7	35,43-44,102,112-125
gameController.js	77.77	65	100	77.77	29-30,46-47,69-70,84-87,99,107-108,124-125
teamController.js	86.2	57.14	100	86.2	85,92-102
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-/Docs/Baseball/Model	96.47	50	95.83	96.47	
UserDao.js	100	100	100	100	
gameDao.js	100	100	100	100	
teamDao.js	89.65	50	87.5	89.65	82,88-89

Test Suites: 2 failed, 4 passed, 6 total
Tests: 3 failed, 67 passed, 70 total
Snapshots: 0 total

Figure 4: Coverage report

5.3.4 Retrospective & Reflection

Fabrizio: Throughout this iteration I didn't come across any major challenges in terms of working with the basic HTML front end, although React has become a bigger challenge than originally anticipated, that being said this week is a much lighter week in terms of course load so I will be able to dedicate more time to trying to implement react. During this iteration I learned how the simplest features found in common website are commonly over looked, So this week I plan on reviewing my two pages and creating a checklist of Requirements in terms of Prof. Ebert's demands and adding other features that were overlooked and implement them.

Dylan: A major challenge I face was how to implement logos into to team in our data base. I originally just had a local file that used multer to store the logos. It worked but only on my local device, and also would not have live updates unless you git pushed every time a team was made. So I was able to use a buffer which is a part of mongo db. Definitly should have started off with this but after some tweaks I got it working and should preform well on a live server. I think for the next iteration I should fully explore what mongo db has to offer before searching it up. I think that this iteration didn't contain much workload but definitely was tedious and can be avoided by planning. It made me learn a lot about images and how they are stored in database and displayed on front end. Defintly my favorite sprint as I saw a lot of progress in this iteration and can really start to see the project come together.

Will: An issue I faced during this was the update user portion. The reason this was so hard was that I ran into an issue revolving reading names. I was unsure of why this occurred but realized I changed a few lines of code in userDao and userController when trying to figure out how to finish the update function. This iteration was a lot easier compared to Iteration 1 but still had its challenges with constantly having to look back at previous code and question why it isn't working now but was working previously.

Zoe: This iteration went pretty well overall, but I did run into some small problems trying to get the add child feature working correctly. It took a while to figure out how to connect everything between the front end and the backend, especially making sure the routes and database were all lined up. I also spent time fixing small things on the landing page and making it easier to navigate, which was fun but a little time-consuming. I learned a lot more about how the backend and frontend work together and how small details can cause big issues if something doesn't match up. Next time I want to plan out my routes and data flow earlier so I don't have to spend as much time troubleshooting later.

5.4 Iteration/Sprint 3

5.4.1 Planning

n[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

Our main plan was to touch up on previous iterations as well as add some final features (such as the rating system and the live chat box).

Fabrizio: U9[Full CRUD] For this Iteration I planned on completing the full CRUD for the ChatBox Feature. I realized this was our Last iteration and wanted to push myself to try something "out of my depth", while having previously working on the game CRUD I felt I could confidently complete this Crud.

Zoe: For this iteration, I planned to complete G0 (Login as Guest), T1 (Beautification/Polishing of Web Design GUI), and ADM13 (Game Type), for a total of 5 story points. My main focus was to enhance the user interface and user flow, because out of our team members I feel I have the strongest design instincts and visual intuition. I wanted to take the lead on elevating the look and feel of the site and ensuring a cleaner and more intuitive navigation experience for all user roles.

Will: I did U5, U6, U7, U8 (CRUD for rating system), and ADM12 (Admin Page) [4-8 for CRUD (I believe it was not worth 8 points so I halved it to 4)] and [1 for Admin Page]. Total = 5 User Story points. I chose these user stories as a rating system and a functioning admin page are essential to the program we want to build and we had not implemented them yet.

Dylan: I planned out that I would do ADM11, M3, U4, C1 which is the 4/4 crud for an invitation system. I created ways to create, read, update, and delete invitations. I think this would account for 8 points in

total. I chose these user stories because i believe it was important to make sure that adding players, adding a coach, and adding a manager was all done through an invite. This way the user would be aware of them being added to a team.

5.4.2 Work Done

Fabrizio: For this iteration I attempted to complete the full CRUD for the chat box feature in our web App. Although I was only able to complete CRD(Create, Read, and Delete), I also worked on creating a proper logout method along with implementing sessions, in order to best utilize the chat features along with the much needed sessions.

Zoe: During this iteration I completed all three stories I originally planned. I finished G0 (Login as Guest), which included creating the guest login and setting access restrictions so that guest users cannot open certain tabs (Teams, Schedule, etc.). This required adjusting the navigation logic and frontend routing. Second, I completed T1, which involved polishing the UI across the landing page and several other pages. I refined spacing, alignment, colors, and button layouts to give the web app a more professional and cohesive feel. I made it so you see the same navigation base across every page. This involvoed writing a css file and implementing that into every html file. Finally, I completed ADM13 (Game Type). On the backend, I added a new game attribute to store the match type and updated the controller and DAO to read and write that value. On the frontend, I added UI labels for each game and implemented a dropdown in the admin interface so admins can select a game's type when creating or editing a match. This feature gives the application clearer structure for displaying playoff and final games.

Will: For this iteration, I completed all 5 stories I set out to do. These include the full CRUD for rating (U5,U6,U7,U8) and the admin page (ADM12). Originally, I believed the rating system would be more challenging and originally said it would be worth 8 points for full CRUD. I realized that it was not nearly as difficult as I thought and realized it should only be worth 4 points at most. The admin page was also very simple to code (which I assumed it would be) and the only factor I was worried about was doing the admin check but coincidentally, I did something similar when doing the rating system so it was very easy. This iteration made me realize that I have learned a lot about javascript and html since the class began. Something I learned had to do with referencing an attribute from a user. For some reason, this caused me a lot of trouble on the previous iterations but after realizing I was overcomplicating it, I rezlied I wasted way too much time trying to brute force my way through grabbing attributes instead of trying to think intuitively about it.

Dylan: During this iteration I was able to complete all my user stories that I had planned. I was able to create a fully functional invitation system that would send invitations to users when they were invited to a team. When adding a child to a team as a player it would invite the child but send an invitation to the parent, setting the invitation as pending, and if accepted change it to accepted or declined if declined. Same was done for adding a coach or changing a manager but the invitation is sent directly to them. I also hid some features on the home screen so not just anyone could make team and edit it. I created an HTML so you can view all your invites and accept or decline them or even delete. Creating the invites was done on editTeam.htlm which I also did alot of work to make it work better with invites. Doing all of these tasks allowed for all my user stories to be fully complete.

5.4.3 Testing Coverage

Fabrizio: For this iteration, I implemented a chat Box Feature that allows users to message one another via our web app. So in terms of testing I created tests for both the chatDao and chatController Files, were I was able to achieve full coverage for both. Below is a screenshot of the testing coverage report:

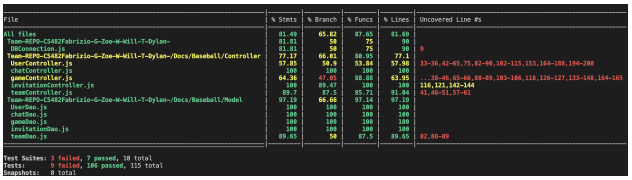
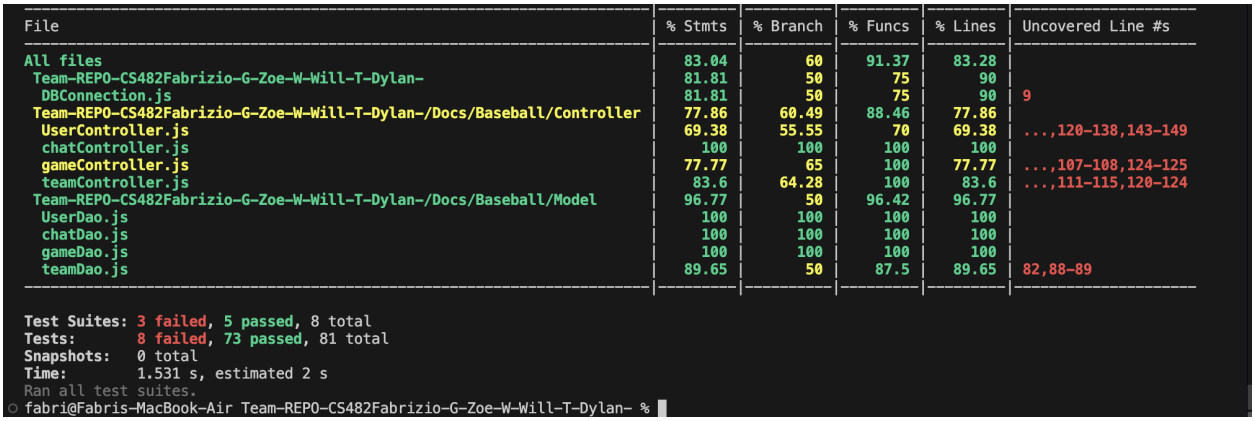


Figure 6: Dylan Testing coverage



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	83.04	60	91.37	83.28	
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-	81.81	50	75	90	
DBConnection.js	81.81	50	75	90	9
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-/Docs/Baseball/Controller	77.86	60.49	88.46	77.86	
UserController.js	69.38	55.55	70	69.38	...,120-138,143-149
chatController.js	100	100	100	100	
gameController.js	77.77	65	100	77.77	...,107-108,124-125
teamController.js	83.6	64.28	100	83.6	...,111-115,120-124
Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan-/Docs/Baseball/Model	96.77	50	96.42	96.77	
UserDao.js	100	100	100	100	
chatDao.js	100	100	100	100	
gameDao.js	100	100	100	100	
teamDao.js	89.65	50	87.5	89.65	82,88-89

Test Suites: 3 failed, 5 passed, 8 total
Tests: 8 failed, 73 passed, 81 total
Snapshots: 0 total
Time: 1.531 s, estimated 2 s
Ran all test suites.
fabri@Fabris-MacBook-Air Team-REPO-CS482Fabrizio-G-Zoe-W-Will-T-Dylan- %

Figure 5: Coverage report

Dylan: For this iteration, I had done testing for invitations for both the controller and dao. Making sure that I got acceptable coverage reports that showed good coverage.

5.4.4 Retroespective & Reflection

Fabrizio: The biggest challenges I faced during this iteration was knowing where to start...In other words having to check over previous work to make sure my perception of the projects progress was up to date. I then realized in order to fufill the CRUD of the CHatbox, sessions needde to be implemented. So i learned how to implement sessions and worked oN creating my chat box, but came across an isssue that took far to much time. AS I singed in as a user and entered messages I realized there wasnt a logout process, so I went back and created a logout. Now After creating that logout route, it just came down to Cleaning up the UI to fit the clients needs and finish up testing.

Zoe: This iteration was a strong mix of visual design work and full-stack development. One of the most meaningful challenges for me was creating and managing our new CSS stylesheet for the UI improvements. I had never written CSS before this sprint, so learning how to structure selectors, understand the box model, manage spacing, and apply consistent styling across pages was a major learning experience. By the end, I felt much more confident using CSS to control layout, typography, and interactive elements, and it became one of my biggest takeaways from this iteration.

Will: In this iteration, the biggest challenge I faced was how ambitious I wanted to be. I was able to make the basic design of the rating system fairly quickly but couldn't decide if I wanted to add more features to it or not. I spent a lot of time trying to change the code to work and look better but every time I tried to add to it, my bugs kept snowballing and I kept getting more lost. Eventually, I committed my original rating system and decided that I should complete the admin page and come back to the rating system later. Because we are allowed to touch up on the code after this iteration, I will make sure to figure out my errors and make the rating system even better. That being said, I believed this iteration truly showed me that I am much better at javascript compared to when I first started.

Dylan: This iteration, was definitely the hardest one as making an invitation system involved using team, user, and invitation to get it working. Having to figure out how store invitations took a while to figure because I was debating between storing it in the user class but ended up making invitations their own class.

Testing the invitation was hard because I ran into conflicts while testing because it uses user and team. To solve the testing issue I just changed the package.json for testing to use –runInBand to run all the tests one at a time and instead of all at the same time. This fixed the issue and my tests then passed. I think things that didnt go according to plan was how much work was needed to my previous HTML documents, so that they could would with invitations. I didn't make the HTML to work with invites and for future interations I would definitely have a plan of user stories for even future iterations that way everything will be made with the next iterations in mind. I learned that projects are not just one step at a time but actually knowing what future steps are so when you come across them, implementation will be a lot easier, because you planned for it.

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project.]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently. How can you do better work in future projects? You may write this as a team or per person (or both)]

Appendix

[Appendix section if needed]